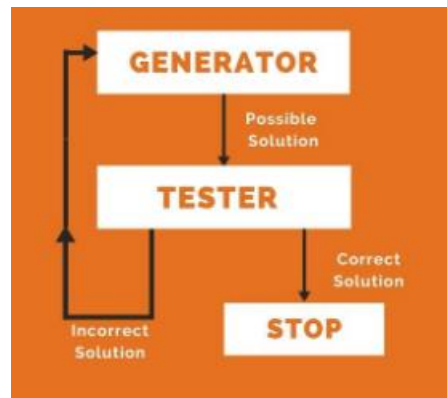


Generate and Test

- Generate and Test Search is a simple problem-solving technique in AI that involves generating possible solutions and testing them to see if they satisfy the given goal or conditions. It's often used in scenarios where a direct path to the solution is not known, so the system explores multiple possibilities until the correct one is found.
- **Generate:** Create possible solutions to the problem. This can be done randomly, systematically, or through heuristic approaches.
- **Test:** Evaluate each generated solution against the goal or criteria to determine if it solves the problem.
- **Success Check:** If a solution meets the goal, the process stops, and the solution is returned. If not, the process continues.
- **Repeat:** Continue generating and testing until a valid solution is found or all possibilities are exhausted.



Why use Generate and Test search?

- **Simplicity:** It's a straightforward approach, easy to implement without needing a deep understanding of the problem's structure.
- **Flexibility:** Can be used for various types of problems, especially when other strategies are complex or unknown.
- **Heuristics Integration:** Heuristics can be added to the generation phase to create more promising solutions.

Limitations:

- **Inefficiency:** It can be slow, as it may generate many invalid solutions, especially with large search spaces.
- **Non-Optimal:** It doesn't guarantee finding the best solution, just a valid one.
- **Randomness:** Depending on how solutions are generated, it might miss some possibilities without exhaustive testing.

Hill climbing

- Hill Climbing is an iterative optimization algorithm used in AI to find a solution by making incremental changes to a single solution and moving toward a direction that improves it.
- The algorithm keeps moving to a better state until no further improvements can be made.

Characteristics:

- **Local Search Algorithm:** Hill climbing searches for the optimal solution by examining only nearby states rather than considering the entire search space.
- **Greedy Approach:** It chooses the best immediate move without considering the bigger picture, leading directly to a higher value.
- **No Backtracking:** Once a move is made, it does not revert or explore past states, which can lead to problems if stuck at a local maximum.
- **Termination:** The algorithm stops when it reaches a peak (maximum value) where no neighbors have a higher value.

Type of Hill Climbing:

- **Simple Hill Climbing:** Chooses the first neighbor that improves the solution.
- **Steepest-Ascent Hill Climbing:** Examines all neighbors and moves to the one with the highest value.
- **Stochastic Hill Climbing:** Chooses a random move among the uphill moves, making the search less deterministic.

Challenges:

- **Local Maxima:** The algorithm can get stuck at a local peak, missing the global maximum.
- **Plateaus:** Flat areas where all neighboring states have the same value can halt progress.
- **Ridges:** Narrow pathways that require specific moves to navigate, which can be problematic for basic hill climbing.

Simple Hill Climbing

- Simple hill climbing is the simplest way to implement a hill climbing algorithm. It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state. It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state.

Algorithm:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
 - If it is goal state, then return success and quit.
 - Else if it is better than the current state then assign new state as a current state.
 - Else if not better than the current state, then return to step2.
- **Step 5:** Exit.

Steepest-Ascent Hill Climbing

- The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors.
- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
- **Step 2:** Loop until a solution is found or the current state does not change.
 - Let SUCC be a state such that any successor of the current state will be better than it.
 - For each operator that applies to the current state:
 - Apply the new operator and generate a new state.
 - Evaluate the new state.
 - If it is goal state, then return it and quit, else compare it to the SUCC.
 - If it is better than SUCC, then set new state as SUCC.
 - If the SUCC is better than the current state, then set current state to SUCC.
- **Step 3:** Exit.

Knowledge Representation

- Knowledge representation and reasoning (KR, KRR) is the part of Artificial intelligence which concerned with AI agents thinking and how thinking contributes to intelligent behavior of agents.
- It is responsible for representing information about the real world so that a computer can understand and can utilize this knowledge to solve the complex real-world problems such as diagnosis a medical condition or communicating with humans in natural language.
- It is also a way which describes how we can represent knowledge in artificial intelligence. Knowledge representation is not just storing data into some database, but it also enables an intelligent machine to learn from that knowledge and experiences so that it can behave intelligently like a human.

What to represent?

- **Object:** All the facts about objects in our world domain. E.g., Guitars contains strings, trumpets are brass instruments.
- **Events:** Events are the actions which occur in our world.
- **Performance:** It describe behavior which involves knowledge about how to do things.
- **Facts:** Facts are the truths about the real world and what we represent.
- **Knowledge-Base:** The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences

Knowledge Representation

ADVANTAGES:

- It is easy to represent and understand as well.
- It require less memory.

DISADVANTAGES:

- It is very hard to differentiate between name and characteristic.

e.g. Virat kohli is a cricketer. Sol: CRICKETER VIRAT KOHLI
KOHLI

- Fails to capture the relationship of the real world facts.

e.g. The sun is rising. Sol: (Rising)(Sun)
(predicate)(argument)

Knowledge Representation

1. Man is mortal.

Sol: Mortal (man)

2. Virat Kohli is a cricketer.

Sol: cricketer(virat kohli)

3. Sun is rising in the east.

Sol: rising(sun, east)

4. Virat Kohli is a left- handed cricketer.

Sol: cricketer(virat kohli, left handed)

1. Today is sunny day.
2. I like robotics.
3. John is a brother of Alex.
4. John loves to play guitar.
5. I scored good marks in AI.

1. today(sunny day)
2. likes(x, robotics)
3. brother(John , alex)
4. love_to_play (John, guitar)
5. scored_ai (x , good)

Knowledge Representations

- We are only going to look at one logical representation scheme, namely, first-order predicate logic. Predicate logic is more powerful than propositional logic as it allows for quantification. Facts are represented as logical propositions and additional information is deduced from these facts using backward chaining or resolution.
- The first step is to convert facts in English to logical propositions called well-formed formulas (wffs). The wffs include the following symbols:
 - Implies: $\rightarrow$
 - And: $\wedge$
 - Or: $\vee$
 - Not: $\sim$
 - For all: $\forall$
 - The exists: $\exists$

- Example: Consider the following facts on Italian history:

1. Marcus was a man.
2. Marcus was a Pompeian.
3. All Pompeians were Roman.
4. Caesar was a ruler.
5. All Romans were either loyal to Caesar or hated him.
6. Everyone is loyal to someone.
7. People only try to assassinate rulers that they are not loyal to.
8. Marcus tried to assassinate Caesar.
9. All men are people.

1. $\text{man}(\text{Marcus})$
2. $\text{Pompeian}(\text{Marcus})$
3. $\forall x: \text{Pompeian}(x) \rightarrow \text{Roman}(x)$
4. $\text{ruler}(\text{Caesar})$
5. $\forall x: \text{Roman}(x) \rightarrow \text{loyal_to}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$
6. $\forall x \exists y: \text{loyal_to}(x, y)$
7. $\forall x \forall y: \text{person}(x) \wedge \text{ruler}(y) \wedge \text{tryassassinate}(x, y) \rightarrow \sim \text{loyal_to}(x, y)$
8. $\text{tryassassinate}(\text{Marcus}, \text{Caesar})$
9. $\forall x: \text{man}(x) \rightarrow \text{person}(x)$

1. All men are mortal.
2. All students like AI.
3. Ram likes to eat all kind of food.
4. All yellow mushrooms are poisonous.
5. When it started raining all student carry umbrella.
6. Some purple mushroom are poisonous.
7. Not all students have AI book.
8. Some students do not have AI book.
9. Mohan eats peanuts and goes to school.
10. Venus is near sun but does not have tail.

1. $\forall x(\text{Man}(x) \rightarrow \text{Mortal}(x))$
2. $\forall x(\text{Student}(x) \rightarrow \text{Likes}(x, \text{AI}))$
3. $\forall y(\text{Food}(y) \rightarrow \text{Eat}(\text{Ram}, y))$
4. $\forall x((\text{Yellow}(x) \wedge \text{Mushroom}(x)) \rightarrow \text{Poisonous}(x))$
5. $\text{Raining}() \rightarrow \forall x(\text{Student}(x) \rightarrow \exists y(\text{Umbrella}(y) \wedge \text{Carry}(x, y)))$ OR
 $\forall x: \text{student}(x) \wedge \text{start}(\text{raining}) \rightarrow \text{carry}(x, \text{umbrella})$
6. $\exists x((\text{Purple}(x) \wedge \text{Mushroom}(x)) \wedge \text{Poisonous}(x))$ OR
 $\exists x: \text{mushroom}(x) \wedge \text{color}(x, \text{purple}) \rightarrow \text{poisonous}(x)$
7. $\neg \forall x(\text{Student}(x) \rightarrow \exists y(\text{Book}(y) \wedge \text{AI}(y) \wedge \text{Have}(x, y)))$
8. $\exists x(\text{Student}(x) \wedge \neg \exists y(\text{Book}(y) \wedge \text{AI}(y) \wedge \text{Have}(x, y)))$
9. $(\text{Eat}(\text{Mohan}, \text{Peanuts}) \wedge \text{GoesToSchool}(\text{Mohan}))$
10. $\text{Near}(\text{Venus}, \text{Sun}) \wedge \neg \text{Tail}(\text{Venus})$

1. Lucy is a professor.
2. All professors are people.
3. Alex is the dean.
4. Deans are professors.
5. All professors consider the dean a friend or don't know him.
6. Everyone is a friend of someone.
7. People only criticize people that are not their friends.
8. Lucy criticized Alex.

1. Professor(Lucy)
2. $\forall x(\text{Professor}(x) \rightarrow \text{Person}(x))$
3. Dean(Alex)
4. $\forall x(\text{Dean}(x) \rightarrow \text{Professor}(x))$
5. $\forall x(\text{Professor}(x) \rightarrow (\text{Considers_Friend}(x, \text{Alex}) \vee \text{Doesn't_Know}(x, \text{Alex})))$
6. $\forall x(\exists y \text{Friend}(x, y))$
7. $\forall x \forall y ((\text{Person}(x) \wedge \text{Person}(y) \wedge \neg \text{Friend}(x, y)) \rightarrow \text{Criticizes}(x, y))$
8. Criticized(Lucy, Alex)

Backward Chaining

- Suppose that given the facts 1 to 9 in the previous section, we want to answer the question ‘Was Marcus loyal to Caesar’? We can prove this using backward chaining.
- Backward chaining matches a clause to the right-hand side of a rule (wff with an implies) and then attempts to prove the clauses on the left-hand of the rule.
- Alternatively, a clause can be matched against a single clause. Since wff 7 has a $\sim$ loyalto on the right-hand side we will attempt to prove $\sim$ loyalto(Marcus, Caesar):

Backward Chaining

$\sim\text{loyalto}(\text{Marcus}, \text{Caesar})$

↓ 7

$\text{person}(\text{Marcus}) \wedge \text{ruler}(\text{Caesar}) \wedge \text{tryassassinate}(\text{Marcus}, \text{Caesar})$

↓ 8

$\text{person}(\text{Marcus}) \wedge \text{ruler}(\text{Caesar})$

↓ 4

$\text{person}(\text{Marcus})$

↓ 9

$\text{man}(\text{Marcus})$

↓ 1

Nil

Using backward chaining we have proved that Marcus is not loyal to Caesar.

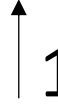
1. John likes all kinds of food.
 2. Apples are food.
 3. Chicken is food
 4. Anything anyone eats and is not killed by is food.
 5. Bill eats peanuts and is still alive.
 6. Sue eats everything Bill eats.
- a) Convert these facts to wffs in predicate logic.
 - b) Using backward chaining prove the “John likes peanuts”.

1. $\forall y(\text{Food}(y) \rightarrow \text{Likes}(\text{John}, y))$
2. $\text{Food}(\text{Apples})$
3. $\text{Food}(\text{Chicken})$
4. $\forall x \forall y ((\text{Eat}(x, y) \wedge \neg \text{KilledBy}(x, y)) \rightarrow \text{Food}(y))$
5. $\text{Eat}(\text{Bill}, \text{Peanuts}) \wedge \text{Alive}(\text{Bill})$
6. $\forall y (\text{Eat}(\text{Bill}, y) \rightarrow \text{Eat}(\text{Sue}, y))$

Nil

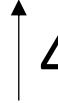


$\forall y(\text{Food}(y) \rightarrow \text{Likes}(\text{John}, y))$



1

$\forall x \forall y ((\text{Eat}(x, y) \wedge \neg \text{KilledBy}(x, y)) \rightarrow \text{Food}(y))$



4

$\text{Eat}(\text{Bill}, \text{Peanuts}) \wedge \text{Alive}(\text{Bill})$



5

$\text{Likes}(\text{John}, \text{Peanuts})$

1. Steve only likes easy courses.
 2. Science courses are hard.
 3. All the courses in the basketweaving department are easy.
 4. BK301 is a basketweaving course.
- a) Convert these facts to wffs in predicate logic.
 - b) Use backward chaining to answer the question “What course would Steve like?”.
- NOTE: Add sentence if necessary.(with logic)

1. $\forall y(\text{Likes}(\text{Steve}, y) \rightarrow \text{Easy}(y))$
 $\forall y(\text{Easy}(y) \rightarrow \text{Likes}(\text{Steve}, y))$
2. $\forall y(\text{Science}(y) \rightarrow \text{Hard}(y))$
3. $\forall y(\text{Basketweaving}(y) \rightarrow \text{Easy}(y))$
4. $\text{Basketweaving}(\text{BK301})$

1. Nil
2. $\forall y(\text{Easy}(y) \rightarrow \text{Likes}(\text{Steve}, y))$
3. $\forall y(\text{Basketweaving}(y) \rightarrow \text{Easy}(y))$
4. $\text{Easy}(\text{BK301})$
5. $\text{Likes}(\text{Steve}, \text{BK301})$

